Verilog 大作业实验报告

实验名称	红绿灯交通灯计时电路				
学生姓名	赵宇哲	学号	2411094	指导老师	杨飞
实验地点	公共实验楼 A 区 306 教室		实验时间	2025-11-5 星期三第三四节	

1 实验项目名称

红绿灯交通灯计时电路

2 必修或选修

必修

3 实验平台

Xilinx Vivado Design Suite 2017.1（64-bit）

4 实验内容及步骤

4.1 设计思路

本设计把红绿灯分成三部分：

- 一部分 `tick_gen` 分频产生“秒”脉冲用于计时；
- 一部分 `traffic_core` 根据时间决定亮灯，按“绿 → 黄 → 红”顺序循环计时，分别维持 30ns、4ns、34ns（模拟现实的 30s、4s、34s）；
- `sevenseg2` 将剩余时间显示在数码管上，顶层 `traffic_light` 负责连接各模块并输出灯光与显示信号。

4.2 设计文件

```
`timescale 1ns/1ps

// ---------------- 1 脉冲分频器 ----------------
module tick_gen #(
  parameter integer CLK_FREQ_HZ = 100_000_000,
  parameter integer TICK_HZ     = 1
)(
  input  wire clk,
  input  wire rst,     // 高有效复位
  output reg  tick     // 单周期脉冲（当 DIVISOR>1 时）
);
  // 防止除法得到 0
  localparam integer DIVISOR_TMP = (TICK_HZ == 0) ? 1 : (CLK_FREQ_HZ / TICK_HZ)
      ;
  localparam integer DIVISOR     = (DIVISOR_TMP == 0) ? 1 : DIVISOR_TMP;

  reg [31:0] cnt;

  always @(posedge clk or posedge rst) begin
    if (rst) begin
      cnt  <= 0;
      tick <= 1'b0;
    end else begin
      if (cnt == DIVISOR-1) begin
        cnt  <= 0;
        tick <= 1'b1;
      end else begin
        cnt  <= cnt + 1;
        // DIVISOR==1 时，相当于 tick 始终为 1，每拍都有效
        if (DIVISOR != 1)
          tick <= 1'b0;
        else
          tick <= 1'b1;
      end
    end
  end
endmodule


// ---------------- 两位数码管扫描 ----------------
module sevenseg2 #(
  parameter integer CLK_FREQ_HZ     = 100_000_000,
  parameter integer SCAN_HZ         = 1000,  // 每位约1kHz
  parameter         SEG_ACTIVE_LOW  = 1,     // 段选低有效
  parameter         DIGIT_ACT_HIGH  = 1      // 位选高有效
)(
  input  wire        clk,
```

```
  input  wire        rst,
  input  wire [7:0]  value,          // 0..99
  output reg  [7:0]  seg,            // {dp,g,f,e,d,c,b,a}
  output reg  [1:0]  duan            // 位选[高位, 低位]
);
  // 扫描 tick
  wire scan_tick;
  tick_gen #(
    .CLK_FREQ_HZ(CLK_FREQ_HZ),
    .TICK_HZ    (SCAN_HZ*2)
  ) u_scan_tg (
    .clk (clk),
    .rst (rst),
    .tick(scan_tick)
  );

  // 位选择: 0 十位, 1 个位
  reg sel;
  always @(posedge clk or posedge rst) begin
    if (rst)            sel <= 1'b0;
    else if (scan_tick) sel <= ~sel;
  end

  wire [3:0] ones = value % 10;
  wire [3:0] tens = (value / 10) % 10;

  // 七段编码 (a..g, 1=亮)
  function [6:0] enc7;
    input [3:0] d;
    begin
      case (d)
        4'd0: enc7 = 7'b1111110;
        4'd1: enc7 = 7'b0110000;
        4'd2: enc7 = 7'b1101101;
        4'd3: enc7 = 7'b1111001;
        4'd4: enc7 = 7'b0110011;
        4'd5: enc7 = 7'b1011011;
        4'd6: enc7 = 7'b1011111;
        4'd7: enc7 = 7'b1110000;
        4'd8: enc7 = 7'b1111111;
        4'd9: enc7 = 7'b1111011;
        default: enc7 = 7'b0000000;
      endcase
    end
  endfunction

  wire [6:0] seg_raw = enc7(sel ? ones : tens);
  wire       dp_raw  = 1'b0;

  wire [6:0] seg_out = SEG_ACTIVE_LOW ? ~seg_raw : seg_raw;
  wire       dp_out  = SEG_ACTIVE_LOW ? ~dp_raw  : dp_raw;
```

```
98
99    always @(*) begin
100     seg = {dp_out, seg_out[6], seg_out[5], seg_out[4],
101                    seg_out[3], seg_out[2], seg_out[1], seg_out[0]};
102     if (DIGIT_ACT_HIGH)
103       duan = sel ? 2'b01 : 2'b10;
104     else
105       duan = sel ? 2'b10 : 2'b01;
106   end
107 endmodule
108
109
110 // ---------------- 红绿灯核心（绿→黄→红） ----------------
111 module traffic_core #(
112   parameter integer RED_S = 34,  // 红灯 34 tick
113   parameter integer YEL_S = 4,   // 黄灯 4 tick
114   parameter integer GRN_S = 30   // 绿灯 30 tick
115 )(
116   input  wire clk,
117   input  wire rst,
118   input  wire tick_sec,       // "秒"脉冲（这里 1 tick = 1ns）
119   output reg  [2:0] light,    // 3'b100=红, 3'b010=黄, 3'b001=绿
120   output reg  [7:0] left_s    // 剩余计数 1..99
121 );
122   localparam S_RED = 2'd0,
123              S_YEL = 2'd1,
124              S_GRN = 2'd2;
125
126   reg [1:0] state;
127
128   // 下一状态：绿→黄→红→绿
129   function [1:0] next_state;
130     input [1:0] s;
131     begin
132       case (s)
133         S_GRN: next_state = S_YEL;
134         S_YEL: next_state = S_RED;
135         S_RED: next_state = S_GRN;
136         default: next_state = S_GRN;
137       endcase
138     end
139   endfunction
140
141   // 装载计数
142   function [7:0] load_time;
143     input [1:0] s;
144     begin
145       case (s)
146         S_RED: load_time = (RED_S>99)?99:RED_S[7:0];
147         S_YEL: load_time = (YEL_S>99)?99:YEL_S[7:0];
148         S_GRN: load_time = (GRN_S>99)?99:GRN_S[7:0];
```

```
        default: load_time = 8'd1;
      endcase
    end
  endfunction

  // 状态机 + 精准计时
  always @(posedge clk or posedge rst) begin
    if (rst) begin
      state  <= S_GRN;              // 复位后从绿灯开始
      left_s <= load_time(S_GRN);   // 30 tick
    end else if (tick_sec) begin
      if (left_s > 1) begin
        left_s <= left_s - 1'b1;
      end else begin
        state  <= next_state(state);
        left_s <= load_time(next_state(state));
      end
    end
  end

  // 输出灯：复位时灯灭
  always @(*) begin
    if (rst) begin
      light = 3'b000;             // 复位期间三灯全灭
    end else begin
      case (state)
        S_RED:  light = 3'b100;
        S_YEL:  light = 3'b010;
        S_GRN:  light = 3'b001;
        default:light = 3'b001;   // 默认绿灯
      endcase
    end
  end

endmodule


// ---------------- 顶层 ----------------
module traffic_light #(
  parameter integer CLK_FREQ_HZ     = 100_000_000,
  parameter integer TICK_HZ         = 1,
  parameter integer SCAN_HZ         = 1000,
  parameter integer RED_S           = 34,
  parameter integer YEL_S           = 4,
  parameter integer GRN_S           = 30,
  parameter         SEG_ACTIVE_LOW  = 1,
  parameter         DIGIT_ACT_HIGH  = 1
)(
  input  wire clk,
  input  wire rst,            // 高有效
  output wire [2:0] light,
```

```
  output wire [7:0] seg,
  output wire [1:0] duan,
  output wire [7:0] seconds_left
);
  wire tick_sec;

  // 在 tb 里把 TICK_HZ 设成 CLK_FREQ_HZ → 1 tick = 1ns
  tick_gen #(
    .CLK_FREQ_HZ(CLK_FREQ_HZ),
    .TICK_HZ    (TICK_HZ)
  ) u_tick (
    .clk (clk),
    .rst (rst),
    .tick(tick_sec)
  );

  traffic_core #(
    .RED_S(RED_S),
    .YEL_S(YEL_S),
    .GRN_S(GRN_S)
  ) u_core (
    .clk     (clk),
    .rst     (rst),
    .tick_sec(tick_sec),
    .light   (light),
    .left_s  (seconds_left)
  );

  sevenseg2 #(
    .CLK_FREQ_HZ   (CLK_FREQ_HZ),
    .SCAN_HZ       (SCAN_HZ),
    .SEG_ACTIVE_LOW(SEG_ACTIVE_LOW),
    .DIGIT_ACT_HIGH(DIGIT_ACT_HIGH)
  ) u_7seg (
    .clk  (clk),
    .rst  (rst),
    .value(seconds_left),
    .seg  (seg),
    .duan (duan)
  );
endmodule
```

Listing 1: traffic_lights.v

4.3 仿真文件

```
`timescale 1ns/1ps
```

```
module tb_traffic_light;

  // 1GHz 时钟，周期 1ns
  localparam integer CLK_FREQ_HZ = 1_000_000_000;
  real CLK_PERIOD_NS = 1.0;   // 1ns

  // 一圈 68ns：绿 30ns，黄 4ns，红 34ns
  localparam integer RED_S = 34;
  localparam integer YEL_S = 4;
  localparam integer GRN_S = 30;

  reg clk = 0;
  reg rst = 1;

  wire [2:0] light;
  wire [7:0] seg;
  wire [1:0] duan;
  wire [7:0] seconds_left;

  // 1GHz 时钟
  always #(CLK_PERIOD_NS/2.0) clk = ~clk;

  // 顶层实例：1 tick = 1ns
  traffic_light #(
    .CLK_FREQ_HZ    (CLK_FREQ_HZ),
    .TICK_HZ        (CLK_FREQ_HZ),   // tick_gen 除数=1
    .SCAN_HZ        (10_000_000),
    .RED_S          (RED_S),
    .YEL_S          (YEL_S),
    .GRN_S          (GRN_S),
    .SEG_ACTIVE_LOW (1),
    .DIGIT_ACT_HIGH (1)
  ) dut (
    .clk          (clk),
    .rst          (rst),
    .light        (light),
    .seg          (seg),
    .duan         (duan),
    .seconds_left (seconds_left)
  );

  // 复位 + 仿真时长：总共 90ns（0~20 复位，20~90 正常）
  initial begin
    rst = 1;
    #(20.0*CLK_PERIOD_NS);  // 0~20ns 复位，高电平
    rst = 0;                // 20ns 释放复位

    #(70);                  // 再跑 70ns，到 90ns 结束
    $display("Simulation done.");
    $finish;
  end
```

```
53    always @(posedge clk) if (!rst) begin
54      if (!((light==3'b000)||(light==3'b100)||(light==3'b010)||(light==3'b001)))
            begin
55        $display("[%0t] WARNING: illegal light=%03b", $time, light);
56      end
57    end
58
59  endmodule
```

Listing 2: traffic_lights.v

4.4 仿真结果

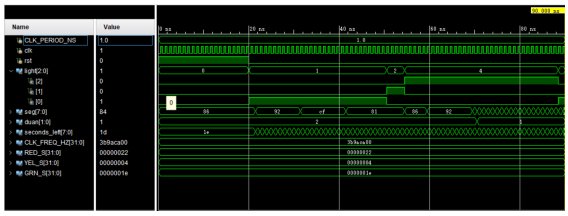


图 1: 红绿灯波形图

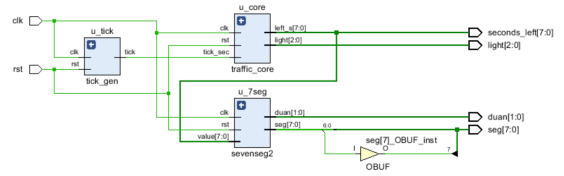


图 2: 红绿灯电路图

5 电路分析

5.1 功能说明

本电路实现一个单向红绿灯控制器。系统按“绿 → 黄 → 红”顺序自动切换灯光，并在数码管上显示每段剩余时间。为方便仿真，30s、4s、34s 分别缩短为 30ns、4ns、34ns，周期共 68ns。

5.2 电路设计

本设计由三个核心模块组成：

(1) **tick_gen（定时脉冲发生器）**

- 输入：clk、rst
- 输出：tick
- 作用：将系统时钟分频，产生稳定的“秒”脉冲（仿真中为 1ns 一跳），供红绿灯计时使用。

(2) **traffic_core（红绿灯主控模块）**

- 输入：clk、rst、tick
- 输出：light[2:0]、left_s
- 作用：内部状态机按“绿 → 黄 → 红”循环，根据 tick 计时。绿灯 30ns、黄灯 4ns、红灯 34ns，并实时输出剩余时间。

(3) **sevenseg2（数码管扫描显示）**

- 输入：clk、rst、left_s
- 输出：seg[7:0]、duan[1:0]
- 作用：将剩余时间以动态扫描方式显示在两位数码管上。

顶层模块 traffic_light 将上述模块连接起来，并输出最终的灯光信号和显示信号。

5.3 电路实现

实现采用 Verilog HDL 编写：

- `tick_gen` 实现时钟分频逻辑，DIVISOR=1 时表示每拍有效。
- `traffic_core` 内部使用有限状态机（FSM），状态依次为 GRN、YEL、RED。每进入一个状态就加载对应的计时值，并逐 tick 递减。
- `sevenseg2` 使用交替位选方式扫描两位数码管，将倒计时值实时显示。
- `traffic_light` 为顶层模块，负责模块例化与端口连接。

5.4 电路验证

根据仿真波形：

- 0–20ns：复位有效，三灯全灭。
- 20–50ns：light=001，绿灯亮，共 30ns。
- 50–54ns：light=010，黄灯亮，共 4ns。
- 54–88ns：light=100，红灯亮，共 34ns。

整个循环共 68ns，完全符合设定的 30ns、4ns、34ns 三个阶段。波形变化与设计逻辑一致，验证了红绿灯的正确性。

6 实验总结

通过本次红绿灯控制器的设计，我进一步熟悉了 Verilog 的模块化思路，学会了如何把计时、状态机、数码管显示等功能拆分后再组合。同时也加深了对时序逻辑、状态切换和仿真调试流程的理解。

在仿真过程中，我遇到过计数不对、波形显示异常问题，通过调整 tick 配置、修改复位逻辑等方式逐步解决，体会到了逐步定位问题的重要性。